

APACHE DORIS 3.0.8

学习笔记

第九章：Catalog、数据湖与联邦查询

主线：Catalog 让 Doris 在不预先拥有数据的情况下理解外部表，并把 FE 规划、BE 并行扫描和向量化计算延伸到 Hive、湖表和关系数据库。

配套编号：notes-09

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

目录	2
本章学习目标	4
9.1 Catalog 到底是什么	4
三段式对象名称	4
Catalog 的三项职责	4
Catalog 与连接器	5
Internal 与 External	5
常用命令	5
9.2 外部表查询的底层链路	5
控制链路 & 数据链路	5
Zero-ETL 的准确含义	6
为什么外表通常不如内表稳定	6
跨Catalog Join 在哪里执行	6
9.3 Hive Catalog	6
Hive 表的三层结构	6
Hive 分区的本质	6
创建与访问	6
Managed 与 External 不要混淆	7
典型瓶颈	7
9.4 Iceberg、Hudi 与 Paimon	7
Iceberg：快照管理文件集合	7
为什么Iceberg适合通用湖仓	7
Hudi：围绕记录更新	7
Paimon：流式更新与LSM	8
选择视角	8
9.5 JDBC Catalog	8
执行链路	8
创建示例	8
适合与不适合	9
9.6 元数据缓存与数据缓存	9
两种缓存不是一回事	9

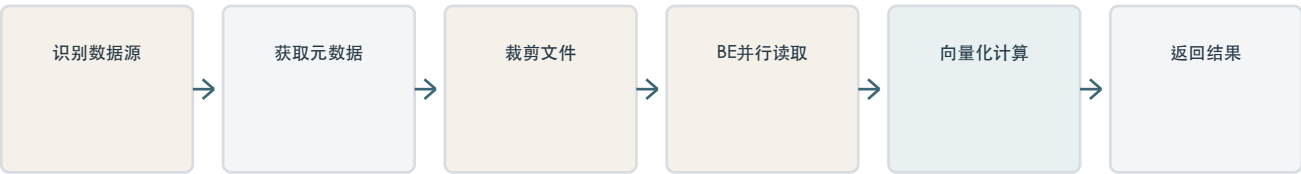
元数据缓存	9
BE数据缓存	9
缓存判断原则	9
9.7 外表查询优化	9
分区条件要可推导	10
列式文件裁剪	10
外表统计信息	10
小文件为什么是规划问题	10
慢查询排查顺序	10
9.8 跨Catalog联邦查询	11
统一入口不等于数据不移动	11
小表Broadcast与大表Shuffle	11
没有跨源全局快照	11
隐性语义边界	11
9.9 Catalog、TVF 与导入内表	11
TVF示例	11
导入或CTAS	12
决策原则	12
9.10 架构设计与诊断	12
冷热分层参考架构	12
诊断树	13
十个反模式	13
最终方法论	13
第九章复习题	13
参考资料	14

本章学习目标

前八章聚焦 Doris 内表的存储、写入、计算、加速与可靠性。本章扩展到数据不进入 Doris 时的分析路径，并建立 Catalog、元数据服务、远程存储、缓存与联邦执行之间的因果链。

问题域	应能回答	关键机制
对象定位	Catalog 是连接器、数据库还是元数据入口？	三段式命名、Internal/External
外表执行	数据不导入为何仍能查询？	FE 取元数据、BE 直读数据
湖表格式	Hive、Iceberg、Hudi、Paimon 差在哪里？	分区、快照、Manifest、Timeline、LSM
联邦查询	跨源 Join 到底在哪里执行？	下推、Broadcast、Shuffle
性能治理	为什么外表慢，先查什么？	裁剪、统计信息、小文件、缓存
架构选择	何时Catalog，何时TVF，何时导入？	冷热分层、负载边界

Catalog 查询主链路



9.1 Catalog 到底是什么

Catalog 是 Doris 内一个有名字、可管理的数据源入口。它描述连接方式和命名空间，帮助 Doris 发现外部库表，但不等于数据副本。

三段式对象名称

catalog.database.table

internal.ads.orders
hive_lake.dwd.order_detail
mysql_biz.crm.customer

Catalog 先确定数据源和访问实现，Database 与 Table 再定位其中的逻辑对象。SWITCH 只改变当前会话的默认 Catalog，不搬迁数据。生产 SQL 使用完整三段式名称更易审计。

Catalog 的三项职责

- 命名空间：解决同名库表属于哪个数据源。
- 连接与认证：保存元数据服务、存储和驱动的连接属性。
- 元数据发现：获得库表、字段、分区、快照和数据位置。

Catalog 与连接器

概念	回答的问题	示例
连接器	底层用什么实现访问	HMS、Iceberg API、JDBC Reader
Catalog	哪一个已命名、已配置的数据源实例	hive_prod、mysql_crm

Internal 与 External

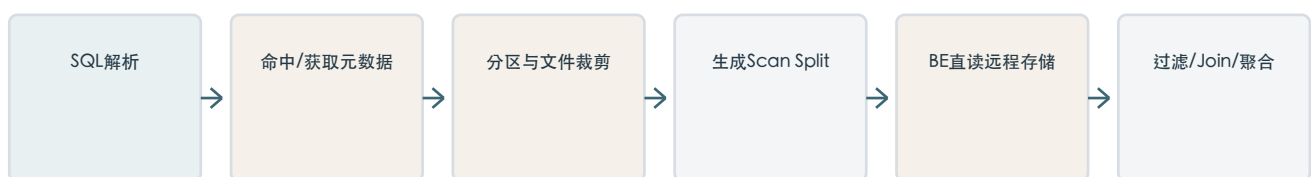
维度	Internal Catalog	External Catalog
固定名称	internal	用户定义
权威元数据	Doris FE	HMS、湖表Catalog、远程DB
数据位置	Doris BE Tablet/Segment	HDFS、S3、远程数据库
布局控制	Doris完整控制	由外部系统决定
删除Catalog	不可删除	只删除映射，不删除源数据

常用命令

```
SHOW CATALOGS;
SWITCH hive_lake;
SHOW DATABASES;
USE dwd;
SELECT * FROM hive_lake.dwd.orders LIMIT 10;
```

9.2 外部表查询的底层链路

控制面负责找数据，数据面负责读数据



控制链路 & 数据链路

metadata path: FE -> HMS / Iceberg Catalog / JDBC metadata
 data path: BE -> HDFS / S3 / remote database

HMS describes files; it does not relay the data bytes.

FE 解析三段式表名，获得 Schema、分区、快照和文件位置，优化器完成裁剪并把文件拆成 Scan Split。多个 BE 并行读取这些 Split，完成解码、过滤、Exchange、Join 与聚合。大量数据不经过 FE。

Zero-ETL 的准确含义

Zero-ETL 表示查询前不必先复制并落库，不表示查询时没有数据传输。远程文件或数据库结果仍要经过网络进入 BE。

为什么外表通常不如内表稳定

external latency $\sim$ metadata RPC + file enumeration + remote IO
+ network + decode + Doris compute
internal latency $\sim$ planning + local replica/cache IO + compute

- Doris 不能完全控制外部文件大小、排序与统计信息。
- 外部元数据服务、网络、对象存储和源数据库都成为依赖。
- 外表不能天然获得 Doris 内表的完整索引、Tablet 与物化布局。

跨Catalog Join 在哪里执行

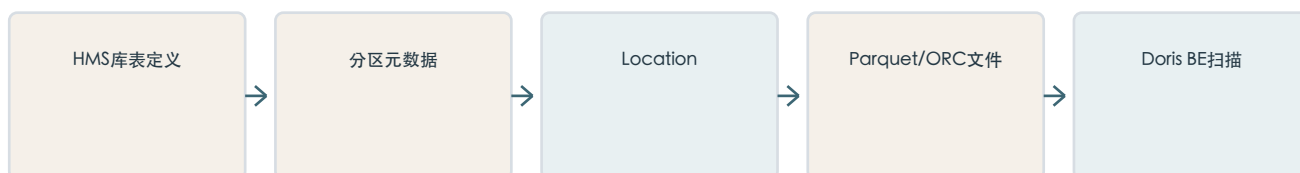
元数据服务只负责描述对象。FE 生成一个统一计划，BE 分别扫描内外部数据，再通过Broadcast或Shuffle完成Join。Catalog不会让HMS替Doris执行Join。

9.3 Hive Catalog

Hive Catalog 通过 Hive Metastore 获得表、分区、格式和 Location，再由 BE 直接读取 HDFS 或对象存储中的文件。

Hive 表的三层结构

Hive 主要定义表，文件系统保存数据



Hive 分区的本质

```
/warehouse/orders/  
dt=2026-08-17/part-00001.parquet  
dt=2026-08-18/part-00001.parquet  
dt=2026-08-18/part-00002.parquet
```

WHERE dt='2026-08-18' 能让优化器只选择一个分区。目录出现在HDFS并不保证HMS已经登记，HMS已更新也不保证Doris缓存立即失效，因此文件状态、Metastore状态与Doris缓存状态必须分别检查。

创建与访问

```
CREATE CATALOG hive_lake PROPERTIES (
  "type" = "hms",
  "hive.metastore.uris" = "thrift://hms:9083",
  "fs.defaultFS" = "hdfs://nameservice1",
  "hadoop.username" = "doris"
);
```

Managed 与 External 不要混淆

Hive Managed/External描述Hive删除和管理数据的语义；Doris External Catalog描述数据源位于Doris之外。无论Hive表是哪一种，对Doris都属于外部表。

典型瓶颈

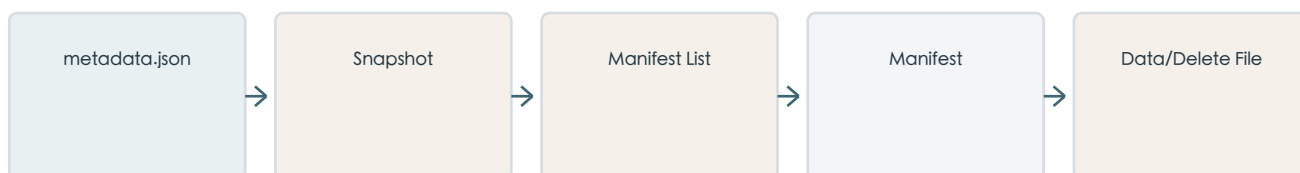
- 小文件导致大量元数据RPC、Scan Split和对象存储请求。
- 百万级分区增加HMS、FE缓存与规划压力。
- HMS Schema与文件物理类型不一致会在读取时失败。
- 查询不带分区条件会把目录裁剪优势完全丢掉。

9.4 Iceberg、Hudi 与 Paimon

现代湖表格式建立在Parquet/ORC与HDFS/S3之上，增加快照、文件集合、更新删除和并发提交语义。它们不是新的文件系统，而是文件之上的表管理层。

Iceberg：快照管理文件集合

Iceberg 不必把目录列表当作当前表的全部事实



每个Snapshot指向一组Manifest，Manifest再描述数据文件、删除文件和列级统计信息。查询先确定快照，再裁剪Manifest与文件。Position Delete记录文件中的行位置，Equality Delete按键值标记删除；它们避免立即重写全部数据，却会增加读取合并成本。

为什么Iceberg适合通用湖仓

- 快照隔离与Time Travel。
- Schema演进与分区演进不必按旧目录规则重写全部数据。
- Manifest可提供比盲目目录枚举更强的文件裁剪。

Hudi：围绕记录更新

类型	写入方式	读取代价
COW	更新时重写Base Parquet	查询简单，写放大较大
MOR	先写增量Log，后台Compaction	Snapshot Query需合并Base与Log

Hudi Timeline记录提交历史，File Group组织Base File和Log。COW更偏读优化，MOR把部分成本从写入转移到查询与Compaction。Doris 3.0可查询COW/MOR并支持相应Snapshot、Read Optimized和Time Travel边界，但不应把它理解为支持所有Hudi增量消费接口。

Paimon：流式更新与LSM

高频小更新通过合并形成稳定文件



Paimon强调主键更新、Changelog、Snapshot与Compaction，常由Flink持续写入。Doris 3.0.8侧主要承担查询分析，不把Doris视为Paimon的完整写入管理引擎。

选择视角

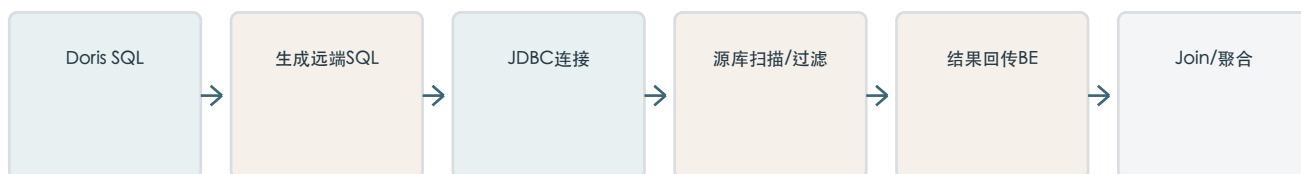
格式	设计重心	常见场景
Iceberg	开放快照、Manifest、演进	多引擎共享的通用分析湖
Hudi	记录级更新、Timeline、COW/MOR	CDC落湖、增量更新
Paimon	Flink、主键、LSM、Changelog	流式湖仓

9.5 JDBC Catalog

JDBC Catalog 是跨库访问和小规模集成工具，不是把单机关系数据库自动变成 MPP 引擎。

执行链路

能下推的条件尽量在源库执行



创建示例

```

CREATE CATALOG mysql_biz PROPERTIES (
  "type" = "jdbc",
  "user" = "query_user",
  "password" = "*****",
  "jdbc_url" = "jdbc:mysql://mysql-host:3306",
  "driver_url" = "mysql-connector-j.jar",
  "driver_class" = "com.mysql.cj.jdbc.Driver"
);
  
```


适合与不适合

适合	不适合
小维表Join、临时核对	扫描十亿行线上交易表
少量数据导入Doris	把JDBC当成CDC同步
迁移过渡和低频联邦分析	两个大型JDBC源之间频繁大Join

无法下推的函数和表达式可能迫使源库返回更多数据，再由Doris过滤。瓶颈仍可能是源库磁盘、执行计划、连接数、JDBC吞吐和网络。要求隔离源库、低延迟或高并发时，应通过CDC把数据同步到Doris内表。

9.6 元数据缓存与数据缓存

两种缓存不是一回事

维度	元数据缓存	数据缓存
主要位置	FE	BE本地磁盘
内容	Schema、分区、快照、文件列表	远程文件数据块
目的	减少HMS/Catalog RPC	减少HDFS/S3重复读取
主要取舍	时效性与元数据压力	命中率、空间与缓存污染
JDBC是否受益	元数据部分可受益	文件Data Cache不适用

元数据缓存

```
REFRESH CATALOG hive_lake;
REFRESH DATABASE hive_lake.dwd;
REFRESH TABLE hive_lake.dwd.orders;
```

缓存减少跨网络请求，但会产生短暂不一致。表名列表、具体表对象、分区列表和文件列表可能属于不同缓存，因此SHOW TABLES与直接SELECT的可见时间可能不同。表级变化优先表级刷新，不要把全Catalog刷新当作日常习惯。

BE数据缓存

```
SET enable_file_cache = true;
```

```
BE config:
enable_file_cache=true
file_cache_path=[{"path":"/data/cache","total_size":53687091200}]
```

第一次查询从远端读取并按文件路径、偏移和块大小缓存，后续命中则从本地SSD读取。它适合热点明确且重复访问的湖表，不适合每次扫描完全不同历史数据的负载。

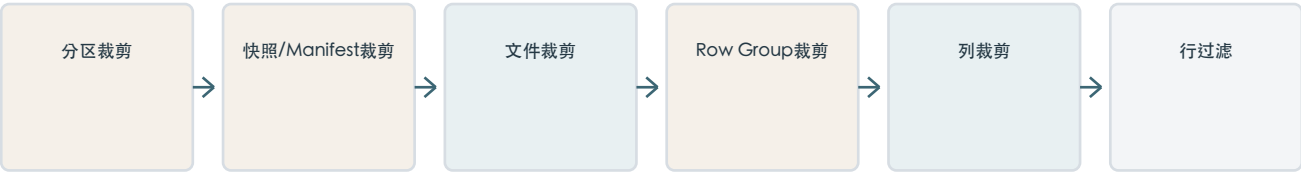
缓存判断原则

- 缓存只能减少重复成本，不能修复无效分区裁剪。
- 缓存容量远小于工作集时，LRU频繁淘汰可能没有收益。
- REFRESH元数据不等同于清理BE文件缓存。

9.7 外表查询优化

外表优化优先减少远程字节数和文件数，然后才是让CPU计算更快。

越早排除无用数据，后续成本越低



分区条件要可推导

```
good:  WHERE dt = '2026-08-18'
risky: WHERE complex_function(dt) = '2026-08-18'

EXPLAIN SELECT ...;
check: partition, predicates, inputSplitNum, totalFileSize
```

列式文件裁剪

SELECT user_id, amount通常明显优于SELECT *。Parquet/ORC还可利用Row Group的min/max、字典和空值统计跳过数据；Iceberg Manifest可在打开数据文件之前继续裁剪。

外表统计信息

CBO需要行数、NDV、min/max和数据分布估算Join顺序及Broadcast/Shuffle。3.0中Hive外表支持自动与采样收集；部分Iceberg、Hudi HMS与JDBC表以手动全量收集为主，External Catalog默认不会自动扫描海量历史数据收集列统计。

```
SHOW TABLE STATS hive_lake.dwd.orders;
ANALYZE TABLE hive_lake.dwd.orders ...;
```

小文件为什么是规划问题

100 GB布局	文件数	主要后果
10 GB/文件	10	并行度可能不足
256 MB/文件	约400	常见合理区间
1 MB/文件	约10万	RPC、Split、调度和IOPS爆炸

128-512 MiB只是常见经验范围，不是Doris硬规则。最好由Spark/Iceberg/Hudi/Paimon/Flink在生产端执行Compaction和文件尺寸治理。

慢查询排查顺序

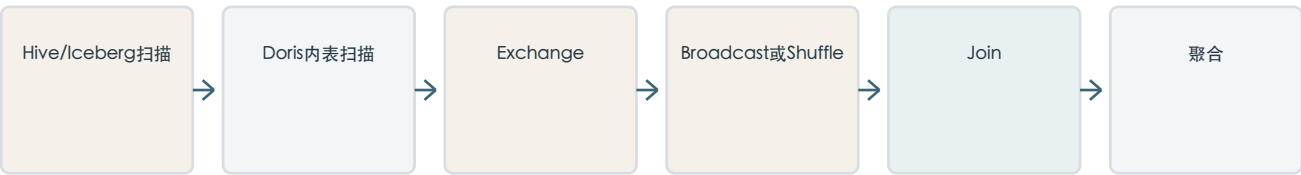
1. 确认分区和Manifest裁剪是否生效。
2. 确认文件数、Split数和扫描字节。
3. 确认列裁剪、谓词下推和统计信息。
4. 确认Join分布、远程IO与Data Cache命中。
5. 最后分析CPU、内存和向量化算子。

9.8 跨Catalog联邦查询

```
SELECT u.region, SUM(o.amount)
FROM hive_lake.dwd.orders o
JOIN internal.dim_users u ON o.user_id = u.id
WHERE o.dt = '2026-08-18'
GROUP BY u.region;
```

统一入口不等于数据不移动

FE生成统一计划，BE执行跨源数据流



小表Broadcast与大表Shuffle

小维表可广播到各BE，与外部大事实表在本地Join。若两边都大，就需要大量源端读取和BE间Shuffle；大型JDBC一侧还可能拖垮源库。因此技术可行不代表适合作为长期服务路径。

没有跨源全局快照

Hive读取10:00的文件集合、MySQL在10:01发生更新时，最终结果未必对应任何全局一致时刻。Catalog统一SQL，但不创造跨系统事务。

隐性语义边界

- 字符串排序规则、大小写和字符集可能不同。
- 时区、Decimal精度和NULL处理可能存在映射差异。
- 任一元数据服务、存储或源数据库故障都会传播到整条查询。
- 强一致结算应先固化到同一快照或同一存储系统。

9.9 Catalog、TVF 与导入内表

维度	Catalog	TVF	Internal Table
使用方式	长期命名对象	SQL中临时指定文件	Doris正式表
是否复制数据	否	否	是
Schema来源	HMS/湖表/远程DB	参数或文件推断	Doris DDL
性能	受外部系统影响	受文件与远程IO影响	通常最稳定
适合	反复查询正式外表	临时检查/导入文件	高并发低延迟服务

TVF示例

```
SELECT * FROM HDFS(
  "uri" = "hdfs://ns/tmp/orders/*.parquet",
  "fs.defaultFS" = "hdfs://ns",
  "hadoop.username" = "doris",
  "format" = "parquet"
);
```

导入或CTAS

```
INSERT INTO internal.ads.orders
SELECT * FROM hive_lake.dwd.orders;

CREATE TABLE internal.ads.orders AS
SELECT * FROM hive_lake.dwd.orders;
```

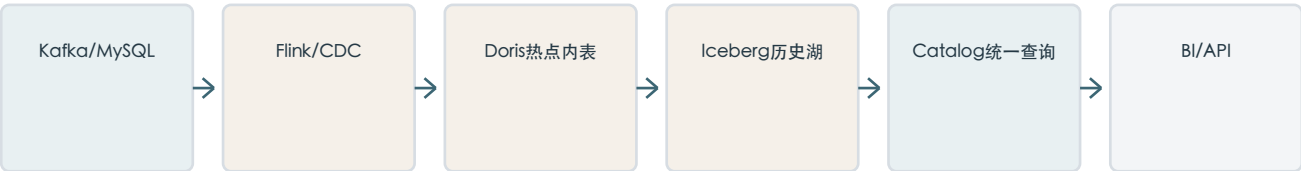
决策原则

- 一次性文件检查选TVF。
- 长期存在、低频或探索型外部表选Catalog。
- 反复查询、高并发、低延迟、需要索引/物化布局时导入内表。
- 无法导入时，再用裁剪、统计信息、Data Cache和源端文件治理优化Catalog路径。

9.10 架构设计与诊断

冷热分层参考架构

实时性能与低成本历史存储各司其职



数据温度	建议位置	原因
热点	Doris Internal Table	高并发、稳定延迟、索引与物化视图
温冷	Iceberg/Hudi/Paimon	低成本长期保存、多引擎共享
临时	HDFS/S3 TVF	无需长期登记
小型外部维表	JDBC或CDC入Doris	按查询频率与源库压力选择

诊断树

症状	优先检查	证据
看不到库表	Catalog配置、认证、HMS、元数据缓存	SHOW CATALOGS、SHOW CREATE CATALOG
规划特别慢	分区/文件数、小文件、元数据服务延迟	EXPLAIN耗时、Split数量
扫描字节过大	分区与谓词裁剪	partition、predicates、totalFileSize
Join爆内存	统计信息和Broadcast/Shuffle	cardinality、Distribution
重复查询仍慢	Data Cache配置与命中、热点是否稳定	Profile、缓存指标、远程IO
只有JDBC慢	下推、源库计划、连接数和网络	源库慢SQL、返回行数

十个反模式

1. 用JDBC全表扫描线上MySQL。
2. 查询Hive大表不带分区条件。
3. 百万小文件只靠增加BE解决。
4. 认为CREATE CATALOG会复制数据。
5. 认为Zero-ETL代表没有网络传输。
6. 把元数据缓存和BE数据缓存混为一谈。
7. 认为外表自动拥有Doris内表性能。
8. 在两个大型JDBC源之间频繁大Join。
9. 忽略外表统计信息却依赖复杂Join。
10. 把核心高并发报表长期绑定到不稳定外部系统。

最终方法论

Catalog 用于消除数据访问边界；Internal Table 用于提供确定的高性能服务。优秀湖仓架构通常让二者按数据温度和查询负载分工，而不是二选一。

第九章复习题

建议按概念边界、控制链路、数据链路、性能代价和架构选择五层回答。

1. Catalog 与连接器有什么区别？
2. 为什么 Hive 表的数据通常不在 Hive Metastore 中？
3. HDFS 出现新分区目录后，Doris 为什么可能看不到？
4. Hive 分区裁剪与 Doris 分桶裁剪有什么区别？
5. Iceberg 为什么需要 Snapshot 和 Manifest？
6. Iceberg Delete File 为什么会增加查询成本？
7. Hudi COW 和 MOR 分别把主要成本放在哪里？
8. Paimon 为什么更适合 Flink 流式湖仓？
9. 为什么 JDBC Catalog 不能把 MySQL 大表自动变成 MPP 查询？
10. 元数据缓存和数据缓存分别位于哪里、缓存什么？

11. 为什么外表统计信息会影响 Broadcast 与 Shuffle 选择？
12. 跨 Catalog 查询为什么不能自动提供全局一致快照？
13. Catalog、TVF 和导入内表分别适合什么场景？
14. 为什么治理小文件通常应优先从数据生产端入手？
15. 如何设计 Doris 热点数据与 Iceberg 历史数据的冷热分层？

参考资料

本文以Apache Doris 3.0.8为基准。3.x页面只采用明确适用于3.0系列的原理与配置；标注为3.1、4.x的新能力未纳入3.0.8课程结论。

- [1] Doris 3.x - Data Catalog Overview
- [2] Doris 3.x - Hive Catalog
- [3] Doris 3.0 - Iceberg Catalog
- [4] Doris 3.0 - Hudi Catalog
- [5] Doris 3.x - Paimon Catalog
- [6] Doris 3.0 - JDBC Catalog
- [7] Doris 3.0 - Metadata Cache
- [8] Doris 3.x - Data Cache
- [9] Doris 3.0 - External Table Statistics
- [10] Doris 3.0 - HDFS TVF
- [11] Release 3.0.8